

GOOD IDEAS:

Semantic Communication + AI

Transmits the information needed to preserve meaning or accomplish a task but not necessarily every original bit.

- **State:** source data (text/image/sensor stream), channel conditions (noise, bandwidth), downstream task requirements.
- **Action:** choice of semantic encoding/compression scheme and transmission rate/power.
- **Reward:** task-success metric (e.g., classification accuracy, task-relevant fidelity) minus bandwidth/energy cost.
- **Hardware:** SDR pair or simulated channel, with edge devices (e.g., RPi) as encoder/decoder nodes.
- **Baseline:** standard compression + channel coding (e.g., JPEG + LDPC) evaluated on the same downstream task.
- **Impact:** could cut bandwidth/energy for IoT/edge use cases where exact reconstruction isn't necessary

AI overview:

Transmit the meaning needed to accomplish a task, not every original bit — cutting bandwidth and energy for edge/IoT links. (DJSC)

- **The idea.** Standard wireless sends data by compressing it (JPEG) and protecting the bits against noise (LDPC error-correction), then rebuilding the file bit-for-bit at the other end — optimizing for *pixel-perfect reconstruction*. This project instead trains a neural **encoder** (sender) and **decoder** (receiver) end-to-end to preserve only what a downstream task needs (e.g. "what class is this image?"), transmitting far less while keeping the task working.
- **Why the learned method genuinely beats the classical one (not a tuning tweak).** Classical coding has no notion of what the bits are *for*, the neural system optimizes directly for task success, an objective JPEG+LDPC structurally cannot express. Theory backs this: Shannon's separation theorem makes the classical compress-then-protect pipeline optimal only for infinitely long messages, but real IoT links send *short* messages over *noisy* channels, exactly the regime where joint learned coding wins. The signature behavior is graceful degradation: classical coding hits a noise threshold and **fails completely (you get nothing)**, while the semantic system just gets **blurrier but still task-correct**.
-
- **Method.** Train an encoder→(differentiable noisy-channel model)→decoder network end-to-end on one narrow task (proposed: **image → classification over an AWGN channel**), minimizing task error *after* channel noise at a fixed bandwidth budget. Baseline: a properly-tuned JPEG + LDPC pipeline on the identical task, SNR, and bandwidth, so the comparison is fair rather than a strawman.
-
- **Scope — de-risked in tiers (simulation-first).** *Tier 1 (the full defensible capstone):* everything above on a **simulated** channel — this is how most published work is evaluated and needs no radio hardware. *Tier 2:* push the encoder's output through a real SDR pair offline to show it survives real impairments. *Tier 3:* live Raspberry-Pi encoder/decoder demo. Tier 1 is built and proven *first*, so the project succeeds even if the hardware tiers don't — the week-one check is confirming the learned system beats the classical baseline on the simulated channel.
- **Success criterion & demo.** Success = **higher task accuracy than JPEG+LDPC at matched bandwidth, especially at low SNR**, shown as an accuracy-vs-SNR plot where the two curves cross. Demo: a live SNR slider driving both pipelines side-by-side on the same image — the classical output shatters past its threshold while the semantic output blurs but the classifier keeps reading it correctly, with the crossover plot updating in real time.

BASIC & SOLVED BUT EASY:

DisasterMesh: AI D2D Offline Emergency Coordination

Create a local communication and decision-support system that continues working when internet connectivity fails.

Agents: message-priority agent, duplicate-request agent, resource-matching agent, safe-route agent and multilingual summarization agent.

Winning demonstration: Several phones form a local network and coordinate simulated requests for food, medicine and rescue.

DyslexiaLens — Adaptive Reading Interface

Transform difficult text based on an individual reader's behaviour.

Signals: reading speed, repeated lines, pauses, pronunciation difficulty and comprehension responses.

Actions: simplify layout, adjust spacing, highlight syllables, provide text-to-speech and generate summaries.

Winning demonstration: Show measurable improvements in reading time or comprehension on a small user study.

SafeScreen: Shoulder-Surfing and Duress Protection

Detect when sensitive content is being viewed by someone other than the owner.

Signals: additional faces, gaze direction, proximity, unusual grip pattern and abnormal navigation behaviour.

Winning demonstration: Automatically blur banking information when another person looks over the user's shoulder.

CodeProof: Private Multi-Agent Debugger

Analyse sensitive code locally without sending the repository to a cloud service.

Agents: static-analysis agent, test-generation agent, dependency agent, security agent and patch-verification agent.

Winning demonstration: Introduce several bugs and show the system generating tests, identifying the root cause and verifying the correction.

Multimodal Scam Detection

Detect scams in SMS messages, emails, WhatsApp screenshots, QR codes, websites, and phone-call transcripts.

Agents: OCR agent, URL-analysis agent, language-manipulation agent, QR-code agent and risk-fusion agent.

Winning demonstration: Present a previously unseen multilingual scam and show the phone identifying the suspicious components while offline.

ReproCheck: Research Reproducibility Auditor

Evaluate whether a research project contains enough information for its results to be reproduced.

Agents: paper-analysis agent, code agent, dataset agent, statistical-method agent and environment agent.

Winning demonstration: Process a paper and repository, then identify missing seeds, dependencies, data-processing steps or unsupported claims.

Similar:

- **★ Few-shot on-device activity/gesture recognition:** a wearable/phone model that personalizes to a *new* user from ~10 examples, entirely on-device. Contribution: on-device adaptation under tiny data, which a cloud-trained fixed model can't do. Demo: enroll a new gesture live, watch it recognize it seconds later. [safe]
 - **★ Privacy-preserving drowsy/distracted-driver detection:** fuse phone IMU (micro-swerves) + front-camera eye state, on-device, no video leaves the phone. Contribution: multimodal fusion + real-time + privacy. Demo: dashboard that flags in real time. [safe]
 - **Acoustic appliance/machine fault monitoring:** one-class/unsupervised anomaly detection on device (fridge, fan, motor). Contribution: no fault labels needed (you can't collect broken-machine data), so it's genuine unsupervised anomaly detection. Demo: introduce a fault, watch the score spike. [safe]
 - **★ On-device keyword spotting with open-set rejection:** tiny wake-word model that also *rejects* out-of-vocabulary words instead of misfiring. Contribution: open-set recognition + sub-100KB footprint. Demo: it wakes on your word, ignores everything else. [safe]
-

UNSOLVED & UNFIT FOR FURTHER WORK:

RL for Dynamic Malloc

arXiv:2410.15492

- **State:** heap fragmentation level, free-list/bin occupancy, incoming request size, recent allocation/deallocation history.
- **Action:** choice of allocation strategy or bin/pool for the incoming request (e.g., best-fit vs. first-fit vs. size-class routing).
- **Reward:** negative fragmentation plus negative allocation latency; large penalty on failed allocation / OOM.
- **Hardware:** standard x86 dev machine; benchmark with mallocbench-style traces or real workloads (e.g., Redis, a long-running service).
- **Baseline:** glibc malloc, jemalloc, tcmalloc under identical workloads.
- **Open question:** inference latency may exceed a standard malloc call; not yet clear. Worth profiling before committing; possible mitigation is restricting RL decisions to large/infrequent allocations only.
- **Impact:** if inference overhead is tolerable, could reduce fragmentation versus heuristic allocators in long-running or memory-constrained systems.

Inference takes up more time than just standard malloc. proved by heatmap

Physics-Guided MagNet

Not proved to be impossible but sensor noise in handheld devices will probably ruin a demo

A hybrid AI system reconstructs hidden low-voltage current-carrying wire paths from sparse, irregular magnetometer scans. Training uses simulated magnetic fields and controlled sensor-rig measurements, while deployment is tested on noisy handheld smartphone scans.

- **Input:** irregular samples containing sensor position, estimated orientation, scan height, and 3-axis magnetic-field measurements ((B_x, B_y, B_z)).
- **Output:** a 2D wire-path occupancy map or graph, estimated current magnitude, and an uncertainty/ambiguity score.
- **AI model:** geometry-aware point-set or graph encoder followed by a grid/graph decoder. The network refines a classical Biot–Savart inverse solution rather than learning the entire reconstruction as an unconstrained black box.
- **Physics integration:** provide the model with physics-solver residuals and enforce forward consistency by comparing the magnetic field generated from the predicted path against the measured field.
- **Training:** generate thousands of synthetic wire layouts with randomized current, sensor height, scan trajectory, orientation error, magnetic bias, noise, and sparse sampling. Fine-tune or calibrate using a smaller real dataset from the controlled rig.
- **Hardware:**
 - Controlled reference setup: external low-noise 3-axis magnetometer mounted on a nonmagnetic XY scanning rig, connected to a microcontroller or SBC for synchronized logging.
 - Deployment setup: Android phone using its magnetometer, accelerometer, and gyroscope during irregular handheld scans.
 - The external sensor should be a sensitive 3-axis magnetometer, not necessarily a “360° angle sensor,” since many 360° sensors are intended to track a

nearby permanent magnet rather than weak current-generated fields.

- **Baseline:** regularized Biot–Savart inversion, physics-only sparse optimization, and a purely data-driven neural reconstruction model under identical train/test layouts.
- **Evaluation:** centerline Chamfer distance, occupancy IoU, current-estimation error, inference latency, and uncertainty calibration. Test on layout-disjoint wire shapes, unseen scan trajectories, different sensor heights, and at least one phone model not used for training.
- **Key ablations:** physics-only vs. neural-only vs. hybrid; with and without orientation input; with and without physics-consistency loss; simulation-only vs. simulation plus real calibration data.
- **Success criterion:** the hybrid model must outperform both the classical inverse solver and the neural-only model on unseen layouts and handheld scans, while flagging most cases where multiple wire paths are physically indistinguishable within the measured sensor noise.
- **Demo:** place interchangeable hidden wire layouts beneath a thin board, scan the surface using a phone, and display a live or near-live overlay of the predicted wire path, current estimate, and uncertainty heatmap. Compare the result against the known hidden layout and the physics-only reconstruction.
- **Open question:** smartphone magnetic bias, uncertain scan height, and trajectory error may dominate the signal from low-current wires. The controlled sensor rig should therefore be built early to establish whether the field is measurable before committing to the full learning pipeline.
- **Impact:** combines inverse problems, structured prediction, sim-to-real learning, and calibrated uncertainty in a low-cost inspection system. Potential applications include educational field visualization, hidden-wire tracing, low-voltage board inspection, and mapping current paths where exact reconstruction may be physically ambiguous.
- **Risk: the project becomes weak as an AI capstone if the neural network is only a cosmetic post-processing step. The contribution must center on learned correction, cross-sensor transfer, generalization to unseen geometries, and reliable ambiguity detection.**

RL-Based Fly-By-Wire

arXiv:2003.08876 · arXiv:2311.13081

(My best idea, but only sounds cool on paper. nightmare to implement, and what *really* classifies as success here anyway? dt-based FBW already is pretty much solved)

- **State:** aircraft attitude (pitch, roll, yaw), airspeed, angular rates, current control-surface positions, pilot input command.
- **Action:** control-surface deflection commands (elevator, aileron, rudder) or flight-control-law adjustments.
- **Reward:** negative tracking error (commanded vs. actual response), penalty for exceeding aerodynamic/structural limits, catastrophic penalty for loss of control.
- **Hardware:** flight simulator or small-scale RC aircraft/drone testbed with autopilot hardware-in-the-loop.
- **Baseline:** classical PID/LQR flight control law under matched simulated conditions.
- **Note:** modern military jets use very deterministic FBW systems, but the military is reportedly more open to RL research here now... could be a timely angle
- **Impact:** explores whether adaptive, learning-based control laws can outperform fixed deterministic ones in maneuverability/robustness, while acknowledging real deployment would require extreme certification and safety guarantees.

Reframed by AI:

RL for Off-Nominal Flight Control Recovery

1. Motivation. Modern fly-by-wire is effectively solved *inside the nominal flight envelope*: a gain-scheduled LQR control law is near-optimal for the linearized dynamics it operates on, and — just as importantly — it is deterministic and certifiable. Trying to beat it there is a losing bet, because even a marginal tracking-error win throws away the predictability that FBW exists to guarantee. So this project deliberately does **not** target nominal control.

2. The reframe (the actual contribution). Aim RL at the regime where classical control breaks down: **post-stall / high-angle-of-attack flight, departure and spin recovery, and adaptation to control-surface damage or actuator failure**. Here the dynamics are strongly nonlinear, no clean linear model exists, gain-scheduled LQR is outside its validated envelope, and recovery is genuinely unsolved. A learned adaptive policy can do something the classical baseline *structurally cannot* — which is the bar this project has to clear to be a real AI contribution rather than cosmetic tuning.

Thermal-Constrained Task Scheduling

RL is NOT better than normal PID scheduling

An RL agent decides CPU frequency and fan speed to clear a compute queue while preventing thermal shutdown.

- **State:** CPU temp, task queue depth/type, current frequency.
- **Action:** discrete combo of DVFS steps and PWM fan settings.
- **Reward:** throughput minus a steep penalty above 85°C and catastrophic termination at 95°C.
- **Hardware:** single-board computer (RPi/Jetson) in an enclosure with a heatsink and PWM fan; workloads generated via stress-ng or matrix-multiply scripts.
- **Baseline:** derive a lumped-parameter RC thermal model from step-response data in MATLAB, then implement a PID thermal governor and fixed-threshold policies to beat.
- **Impact:** proves that pre-emptive, look-ahead RL can outperform reactive thermal management, directly addressing data-center and mobile thermal-throttling challenges.

Semantic Communication + AI

Transmits the information needed to preserve meaning or accomplish a task but not necessarily every original bit.

- **State:** source data (text/image/sensor stream), channel conditions (noise, bandwidth), downstream task requirements.
- **Action:** choice of semantic encoding/compression scheme and transmission rate/power.
- **Reward:** task-success metric (e.g., classification accuracy, task-relevant fidelity) minus bandwidth/energy cost.
- **Hardware:** SDR pair or simulated channel, with edge devices (e.g., RPi) as encoder/decoder nodes.
- **Baseline:** standard compression + channel coding (e.g., JPEG + LDPC) evaluated on the same downstream task.
- **Impact:** could cut bandwidth/energy for IoT/edge use cases where exact reconstruction isn't necessary

RL-Based Cache Eviction

Conservative offline-RL cache-eviction system trained strictly offline on logged access traces and deployed only as a frozen policy inside a real serving path.

- **State:** per-object access features (recency, frequency, size, TTL), current cache occupancy.
- **Action:** evict/keep decision per object, or a ranking over eviction candidates.
- **Reward:** hit-rate improvement computed offline via trace replay (since the policy is frozen at deployment, there's no live exploration signal).
- **Hardware:** Dockerized HTTP caching reverse proxy (Redis eviction-shim as backup) on a Raspberry Pi node (~₹4k, satisfies the hardware clause).
- **Baseline:** LRU, LFU, or ARC eviction policies on the same traces.
- **Impact:** **mostly theoretical and risky!! no strong prior that it beats classic heuristics; the real contribution would be showing offline RL can be safely deployed as a frozen policy in a live serving path without exploration risk.**